

# Ders 3 Çalışma Özeti

## Ders 3 Çalışma Özeti

### Genel Konular

- Mobil uygulama geliştirme yaklaşımları ve sınıflandırılması
  - Temel geliştirme yaklaşımları: Native (yerel) geliştirme, Cross-platform (çapraz platform) geliştirme, Hybrid (melez) uygulama geliştirme, Mobile Web geliştirme ve Progressive Web App (PWA).
  - Native geliştirme: Google (Android için) ve Apple (iOS için) tarafından sağlanan Software Development Kit (SDK) kullanılarak geliştirilen yazılımlar; user experience ve user interface açısından en yüksek memnuniyeti sağlayan yöntem.
  - iOS tarafında Swift ve Objective-C ile; Android tarafında Kotlin, Java, C ve C++ ile native geliştirme yapılabilmesi.
  - Android'e özel Native Development Kit (NDK): SDK'nın yanına ek olarak Google'ın sunduğu, daha alt seviye program geliştirmeyi ve C dili ile yazmayı kolaylaştıran, özellikle gömülü sistem tarafında katkı sağlayan araç.
- Cross-platform geliştirme araçları ve kategorileri
  - Yaygın cross-platform araçları: Flutter, Xamarin, Ionic, React Native, PhoneGap, Titanium, Appcelerator.
  - Son dönemin parlayan yıldızları: React Native ve Flutter.
  - İki kategoriye ayırım: web teknolojilerini kullananlar (React Native: JavaScript, HTML, CSS) ve web teknolojileri kullanmayanlar (Flutter: Dart, Xamarin: C#).
  - "Write once, run everywhere" (bir kere yaz, her yerde çalıştır) felsefesi; kodun farklı platformlara dönüştürülmesi.
  - Unity oyun geliştirme platformu da cross-platform olarak değerlendirilebilir.
- Cross-platform'un avantajları ve dezavantajları
  - Avantaj: tek kod tabanı ile birden fazla platforma uygulama geliştirme; UI bileşenlerinin neredeyse native gibi üretilebilmesi.
  - Dezavantaj: cross-platform framework'ü ilgili platforma native özellikleri sağlamadysa bazı donanım özelliklerini desteklememesi; performans, kod uzunluğu ve enerji tüketimi açısından native'e göre geri kalma potansiyeli.
  - Derleyici kalitesinin önemli olduğu; iyi bir derleyici daha iyi makine dili kodu üretir, ancak arada her zaman bir dönüştürücü katmanı vardır.
- Hybrid uygulama geliştirme
  - Cross-platform'un altındaki bazı araçlarla hybrid app geliştirmenin mümkün olduğu.
  - Web kit üzerinde çalışan uygulamalar; container içinde paketlenerek sanki native gibi markete yüklenme.
  - Hybrid uygulamaların web ortamının güvenlik zayıflıklarına tabi olması.
- Mobil web geliştirme
  - Bilgisayar için web uygulaması geliştirmekten tek farkı: responsive (duyarlı) tasarım.
  - Responsive tasarım: HTML5 ile yazılan uygulamanın her türlü cihazda (telefon, tablet, PC) düzgün görüntülenmesi.
  - Avantajlar: kurulum gerektirmemesi, uygulama indirme zorunluluğunun olmaması, arka planda bilgi erişimi yapılmaması.
  - Ticaret uygulamaları için en güzel örnek olduğu.
  - Dezavantaj: internet bağlantısı olmadan çalışmama.
- Progressive Web App (PWA)
  - Mobil web geliştirmeden çok büyük farkı olmayan, modern web teknolojileriyle geliştirilen uygulamalara verilen isim.

- HTML, CSS ve JavaScript ile geliştirilir ve bir tarayıcı üzerinden kullanılır.
- Tüm tarayıcılarla uyumlu çalışabilen, belli standartlara uygun web uygulamaları oluşturma hedefi.
- Responsive tasarımın ötesinde, tarayıcı farklılıklarını minimize etme amacı.
- Geliştirme yaklaşımlarının karşılaştırılması
  - Quality of user experience (kullanıcı deneyimi kalitesi)
  - Quality of applications (uygulama kalitesi)
  - Potential users (potansiyel kullanıcı sayısı)
  - App development cost (geliştirme maliyeti)
  - Güvenlik seviyesi
  - Güncellenebilirlik
  - Desteklenebilirlik (supportability)
  - Markete ulaşma süresi (time to market)
  - Native: üstün UX, yüksek uygulama kalitesi, platforma özel kullanıcı kitlesi, yüksek maliyet, üstün güvenlik.
  - Mobile Web: orta UX, orta uygulama kalitesi, geniş kullanıcı kitlesi, düşük maliyet, tarayıcı güvenliğine bağlı.
  - Cross-platform: orta-iyi UX, orta-düşük uygulama kalitesi, geniş kullanıcı kitlesi, değişken maliyet, gelişen güvenlik.
- Cross-platform framework tasarım kriterleri
  - Kolay kodlanabilir programlama dili
  - Çoklu mobil platform desteği
  - Zengin kullanıcı arayüzü
  - Güvenlik
  - Düşük güç tüketimi
  - Yerleşik özelliklere erişim (accessing built-in features)
  - Backend iletişim desteği
- Frontend ve Backend kavramları
  - Frontend: kullanıcının yüz yüze olduğu, isteklerini kontrol eden ve gönderecek mesajları yöneten kısım.
  - Backend: arka planda çalışan motor, sunucu tarafı organizasyonu.

## Hocanın Özellikle Vurguladığı Kısımlar

- Cross-platform seçerken platformun iyi belirlenmesi gerektiği
  - “Hangi cross platformla çalışacağınızı iyi belirlemeniz lazım” çünkü çeşitlilik çok fazla; PhoneGap 8-9 senedir var ama artık eski kalmış.
  - Cross-platform seçimi uygulama kalitesini doğrudan etkiler.
- Time to market kavramının yanıltıcı olabileceği
  - “Time to market” ile kastedilen geliştirme süresi değil, kullanıcılara ulaşma süresidir.
  - Cross-platform’da uygulama hem hızlı geliştirilir hem de market üzerinden doğrudan kullanıcıya ulaşır; bu büyük avantajdır.
  - Mobile web’de ise milyonlarca web sayfası arasından fark edilmek zor olduğu için kullanıcıya ulaşma zaman alır.
- Güvenlik konusunda farkındalık
  - Native uygulamalar SDK ile garanti altına alınmış, güvenlik açısından daha iyi.
  - Browserlarda çalışan uygulamalar (mobile web ve hybrid) her zaman bir tık daha saldırıya açık.
- Sınıf ortamında cevap verememenin notu etkileyeceği uyarısı
  - Hoca, derste soru sorduğunda cevap alamadığı öğrencilerin notlarını olumsuz etkileyeceğini açıkça belirtmiştir.
  - Sınıf mevcudunun derse aktif katılımı konusunda ısrarcı tutum takınmıştır.
- E-ticaret uygulamalarının native tercih etme eğilimi
  - “E-ticaret firmalarının bir şekilde native tarafa da kaydığını görüyoruz son dönemde” çünkü güvenlik ve hızlı işlem yapma ihtiyacı.
  - Fiziksel mağazası olmayan firmaların ise mobile web’i tercih edebildiği; bunun prestij ve marka ihtiyacıyla ilgili olduğu.

## Kısa Tekrar Notları

- Mobil geliştirme yaklaşımları: Native, Cross-platform, Hybrid, Mobile Web, PWA.
- Cross-platform örnekleri: Flutter (Dart), Xamarin (C#), React Native (JavaScript), PhoneGap, Ionic, Titanium, Unity.
- Native diller: iOS - Swift, Objective-C; Android - Kotlin, Java, C/C++.
- "Write once, run everywhere" = cross-platform felsefesi.
- PWA = tüm tarayıcılarla uyumlu, responsive web uygulaması.
- Mobil web'in en büyük dezavantajı: internet bağlantısı olmadan çalışmama.
- Time to market = kullanıcıya ulaşma süresi (geliştirme süresi değil).
- Cross-platform framework gereksinimleri: kolay dil, çoklu platform, UI, güvenlik, enerji, native özellik erişimi, backend iletişimi.
- Karşılaştırma kriterleri: UX kalitesi, uygulama kalitesi, potansiyel kullanıcı, maliyet, güvenlik, güncellenebilirlik, supportability, time to market.

## Detaylı Açıklamalar

Dersin başlangıcında hoca, geçen haftanın mobil uygulama geliştirme tekniklerine ayrıldığını hatırlatmıştır. Bu hafta öğrencilere bir ödev verileceği, ancak bunun kodlama ödevi değil bir araştırma ödevi olacağı belirtilmiştir. Önümüzdeki haftadan itibaren yavaş yavaş kodlama işine geçileceği, Android Studio'nun en son sürümünün indirilebileceği, geliştirilecek API olarak Android 8.0 (API 27 civarı) belirlenebileceği söylenmiştir. Bu kurulumların önümüzdeki hafta itibarıyla konuşulacağı vurgulanmıştır.

Dersin ana konusu olan mobil uygulama geliştirme yaklaşımlarına geçildiğinde önce native development tanımlanmıştır. Native development, ilgili kurumun (Google ve Apple) kendi geliştirdikleri Software Development Kit kullanılarak geliştirilen tüm yazılımları kapsar. Bu yöntemin user experience ve user interface açısından en yüksek memnuniyeti sağladığı, ilgili işletim sisteminin ve telefonun yeteneklerini en iyi kullanma alternatifi olduğu belirtilmiştir. Ancak her platform için ayrı ayrı geliştirme yapılması gerektiği, bu nedenle maliyetli olduğu söylenmiştir. iOS tarafında Swift ve Objective-C, Android tarafında ise Kotlin, Java, C ve C++ ile geliştirme yapılabileceği açıklanmıştır. Android'e özel Native Development Kit (NDK) tanıtılmış, bunun SDK'nın yanına ek olarak Google'ın sunduğu, daha alt seviye program geliştirmeyi sağlayan, C dili ile yazmayı kolaylaştıran ve özellikle gömülü sistem tarafında katkı sağlayan bir araç olduğu belirtilmiştir.

Cross-platform geliştirme konusuna geçildiğinde öğrencilerden geri bildirim alınarak ilerlenmiştir. Bir öğrenci Flutter, Xamarin, Ionic saymış, başka bir öğrenci React Native'i eklemiştir. Hoca, bunların en temel bilinen cross-platform araçları olduğunu, son dönemin parlayan yıldızlarının React Native ve Flutter olduğunu belirtmiştir. PhoneGap, Titanium, Appcelerator da diğer bilinen araçlar olarak sıralanmıştır. Cross-platform araçları iki kategoriye ayrılmıştır: web teknolojilerini kullananlar (React Native: JavaScript, HTML, CSS) ve web teknolojileri kullanmayanlar (Flutter: Dart). Xamarin'in C# kullandığı, Flutter'ın Dart diliyle ön plana çıktığı ve React Native'in önemli bir rakibi olduğu vurgulanmıştır. Bir öğrencinin Unity sorusu üzerine, Unity'nin özellikle oyun geliştirme noktasında kullanıldığı ve cross-platform olarak değerlendirilebileceği, ancak ne hybrid, ne mobile, ne de native olarak tam sınıflandırılmayacağı belirtilmiştir.

Cross-platform'un "Write once, run everywhere, anywhere" felsefesiyle çalıştığı, tek bir kod parçası ile birden fazla platform için geliştirme yapılabildiği açıklanmıştır. Kodun üretilmesinin arka planda çok iyi bir kütüphane ve yazılım altyapısı gerektirdiği, çünkü hem Android'e hem iOS'a çeviri yapıldığı belirtilmiştir. React Native ve Flutter'ın UI bileşenlerini neredeyse native gibi üretebildiği, eskiden cross-platform'ın en önemli sıkıntılarından biri olan bu görsel farkın artık çok azaldığı söylenmiştir. Ancak cross-platform'un bazı donanım özelliklerini framework sağlamadıysa desteklemediği, performans, kod uzunluğu ve enerji tüketimi açısından native'e göre geri kalabildiği vurgulanmıştır. Hoca bunu bir derleyici benzetmesiyle açıklamıştır: kod makina diline çevrilirken arada compiler giriyor, ne kadar iyi olursa olsun sonuçta bir dönüştürücü var ve alt seviye dile yazılıysa daha performanslı olurdu. Performans farkı kullanıcıya yansımıyorsa (örneğin 10 ms vs 15 ms) cross-platform'un native ihtiyacını ortadan kaldıracabileceği belirtilmiştir.

Hybrid uygulama konusu ele alınmıştır. Cross-platform'un altındaki bazı araçlarla web kit

üzerinde çalışan hybrid uygulama geliştirmenin mümkün olduğu, container içinde paketlenen uygulamanın sanki native gibi markete yüklenebildiği açıklanmıştır. Ancak web kit üzerinde çalıştığı için web ortamının güvenlik zayıflıklarına tabi olduğu, native'in SDK ile garanti altına alınmış güvenliğinin aksine browserlarda çalışan uygulamaların bir tık daha saldırıya açık olduğu vurgulanmıştır.

Mobile web geliştirme, bilgisayar için web uygulaması geliştirmekten farklı bir kavram olarak ele alınmıştır. Tek farkın "responsive" (duyarlı) tasarım olduğu, bir öğrencinin bu konuyu doğru tespit etmesi üzerine açıklanmıştır. Responsive tasarımın hangi cihazda çalışıyorsa kendini o cihazın formunda görüntüleme yeteneği olduğu, HTML5 ile yazılan uygulamanın her türlü cihazdan erişilebilir hale geldiği belirtilmiştir. Mobil web'in en büyük avantajları: uygulama indirme zorunluluğunun olmaması, kurulum sırasında arka planda bilgi erişimi yapılmaması, kurulum gerektirmemesi. E-ticaret uygulamalarının bu yaklaşıma en güzel örnek olduğu söylenmiştir. Ancak native uygulama prestijinin de önemli olduğu, birçok firmanın bu nedenle native uygulama da geliştirdiği vurgulanmıştır. Dezavantaj olarak internet bağlantısı olmadan çalışmama, yani telefon üzerinde uygulamayı çalıştırma imkanı olsa bile internet olmadan işlem yapılamaması belirtilmiştir.

Progressive Web App kavramı açıklanmıştır. Mobile web development'tan çok büyük bir farklılığı olmadığı, modern web teknolojileriyle geliştirilen uygulamalara verilen isim olduğu belirtilmiştir. PWA'nın tüm tarayıcılarla uyumlu çalışabilen, belli standartlara uygun web uygulamaları oluşturma hedefinde olduğu, responsive tasarımın ötesinde tarayıcı farklılıklarını minimize etme amacı taşıdığı vurgulanmıştır. Bir öğrencinin "progressive" kelimesinin neden kullanıldığı sorusuna hoca, ileri derece anlamında responsive tasarımın ötesine geçildiğini, tüm browserlarla uyumlu çalışma hedefinin ifade edildiğini söylemiştir.

Geliştirme yaklaşımlarını karşılaştırmak için kullanılan kriterler hoca tarafından öğrencilere sorularak belirlenmiştir. Bir öğrenci hız ve erişim, başka bir öğrenci platformlar ve güvenlik/optimizasyon, bir başkası markete dağıtım süreleri ve geliştirme maliyetleri, user experience demidir. Hoca bu kriterleri derleyerek şu listeyi oluşturmuştur: Quality of user experience, Quality of applications, Potential users, App development cost, Güvenlik düzeyi, Güncellenebilirlik, Supportability (desteklenebilirlik), Markete ulaşma süresi (time to market). Bu kriterler üç yaklaşım (Native, Mobile Web, Cross-platform) için doldurulmuştur. Native'in UX açısından "excellent", Mobile Web'in "tatminkar" (responsive ve progressiv design ile), Cross-platform'ın "very good" (eskiden "not as good as native apps" denirdi ama artık çok tatminkar deneyimler elde ediliyor) olduğu belirtilmiştir. Time to market kavramının özellikle vurgulanması dikkat çekicidir: hoca, "time to market" ifadesinin geliştirme süresi değil, kullanıcılara ulaşma süresi olduğunu açıkça belirtmiştir. Cross-platform'da uygulama hem hızlı geliştirilir hem de markete konduğu için tek noktadan ulaşılabilir; mobile web'de ise milyonlarca web sayfası arasından fark edilmek zor olduğu için Google aramalarında ön plana çıkmak zaman alır.

Hoca, dersin ilerleyen bölümlerinde bir cross-platform framework tasarımı için nelere dikkat edilmesi gerektiğini sormuştur. Öğrencilerden gelen cevaplar: kolay kodlanabilir programlama dili, çoklu mobil platform desteği (responsive değil, multiple platform support), zengin UI, güvenlik, düşük enerji tüketimi, native özelliklere erişim (night mode, sensör verisi, kamera) şeklinde olmuştur. Backend iletişim desteği de eklenmiştir. Frontend ve backend kavramları kısaca açıklanmıştır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.